

Exercise 4.14

$$\frac{(\text{value-of } exp_1 \ \rho \ \sigma_0) = (val_1, \sigma_1) \quad l \notin dom(\sigma_1)}{(\text{value-of } (\text{let-exp } var \ exp_1 \ body) \ \rho \ \sigma_0) = (\text{value-of } body \ [var = l]\rho \ [l = val_1]\sigma_1)}$$

Exercise 4.15

Variables in the environment are bound to references which are plain integers.

Exercise 4.16

Assume the initial environment p is empty.

```
(value-of <let times 4 = 0 in begin set..> p)
store = ()
```

```
(value-of <begin set..> [times4=0]p)
store = ((0 (num-val 0)))
```

```
(value-of <(times4 3)> [times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p))))
```

```
(apply-procedure (proc-val <proc (x)..>) (num-val 3))
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3)))
```

```
(value-of <if zero(x)..> [x=1][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3)))
```

```
(value-of <-((times4 -(x,1)), -4)> [x=1][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3)))
```

```
(apply-procedure (proc-val <proc (x)..>) (num-val 2))
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3)))
```

```
(value-of <if zero(x)..> [x=2][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3))
          (2 (num-val 2)))
```

```
(value-of <-((times4 (-x,1)), -4)> [x=2][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3))
          (2 (num-val 2)))
```

```
(apply-procedure (proc-val <proc (x)..>) (num-val 1))
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3))
          (2 (num-val 2)))
```

```
(value-of <if zero(x)..> [x=3][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3))
          (2 (num-val 2))
          (3 (num-val 1)))
```

```
(value-of <-((times4 -(x,1)), -4)> [x=3][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
          (1 (num-val 3))
          (2 (num-val 2))
          (3 (num-val 1)))
```

```
(apply-procedure (proc-val <proc (x)..>) (num-val 0))
```

```
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
         (1 (num-val 3))
         (2 (num-val 2))
         (3 (num-val 1)))
```

```
(value-of <if zero(x)..> [x=4][times4=0]p)
store = ((0 (proc-val (<proc (x)..> [times4=0]p)))
         (1 (num-val 3))
         (2 (num-val 2))
         (3 (num-val 1))
         (4 (num-val 0)))
```

```
-(-(- (0,-4) ,-4) ,-4)
```

```
-(- (4,-4) ,-4)
```

```
-(8,-4)
```

12

Exercise 4.17

$$\begin{array}{c}
 (\text{value-of } \text{exp}_1 \ \rho \ \sigma_0) = (\text{val}_1, \sigma_1) \\
 \vdots \\
 (\text{value-of } \text{exp}_n \ \rho \ \sigma_{n-1}) = (\text{val}_n, \sigma_n) \\
 l_1, \dots, l_n \notin \text{dom}(\sigma_n)
 \end{array}
 \hline
 \begin{array}{l}
 (\text{value-of } (\text{let-exp } \text{var}_1 \dots \text{var}_n \ \text{exp}_1 \dots \text{exp}_n \ \text{body}) \ \rho \ \sigma_0) \\
 = (\text{value-of } \text{body} \ [\text{var}_1 = l_1] \dots [\text{var}_n = l_n] \rho \ [l = \text{val}_1] \dots [l_n = \text{val}_n] \sigma_n)
 \end{array}$$

$$\begin{array}{c}
(\text{value-of } \text{exp}_1 \ \rho \ \sigma_0) = (\text{val}_1, \sigma_1) \\
(\text{value-of } \text{exp}_2 \ \rho \ \sigma_1) = (\text{val}_2, \sigma_2) \\
\vdots \\
(\text{value-of } \text{exp}_n \ \rho \ \sigma_{n-1}) = (\text{val}_n, \sigma_n) \\
\hline
(\text{value-of } (\text{call-exp } \text{exp}_1 \ \text{exp}_2 \dots \text{exp}_n) \ \rho \ \sigma_0) \\
= (\text{apply-procedure } (\text{expval} \rightarrow \text{proc } \text{val}_1) \ \text{val}_2 \dots \text{val}_n \ \sigma_n)
\end{array}$$

$(\text{apply-procedure } (\text{procedure } \text{var}_1 \dots \text{var}_n \ \text{body} \ \rho) \ \text{val}_1 \dots \text{val}_n \ \sigma)$
 $= (\text{value-of } \text{body} \ [\text{var}_1 = l_1] \dots [\text{var}_n = l_n] \rho \ [l_1 = \text{val}_1] \dots [l_n = \text{val}_n] \sigma)$

where $l_1, \dots, l_n \notin \text{dom}(\sigma)$.

;; the-grammar

(expression

 ("let" (arbno identifier "=" expression) "in" expression)
 let-exp)

(expression

 ("proc" "(" (arbno identifier) ")" expression)
 proc-exp)

(expression

 ("(" expression (arbno expression) ")")
 call-exp)

;; value-of

(let-exp (vars exps body)

 (let ((vals (map (lambda (exp)
 (value-of exp env))

```

                                exps)))
      (value-of body
        (extend-env* vars
                      (map newref vals)
                      env))))
(proc-exp (vars body)
  (proc-val (procedure vars body env)))
(call-exp (rator rands)
  (let ((proc (expval->proc (value-of rator env)))
        (args (map (lambda (rand)
                      (value-of rand env))
                    rands)))
    (apply-procedure proc args)))

;; apply-procedure : Proc * Listof(ExpVal) -> ExpVal
(define apply-procedure
  (lambda (proc1 vals)
    (cases proc proc1
      (procedure (vars body saved-env)
        (value-of
          body
          (extend-env* vars
                      (map newref vals)
                      saved-env))))))

(define-datatype proc proc?
  (procedure
    (bvar (list-of symbol?))
    (body expression?)
    (env environment?)))

;; environment datatype

```

```

(extend-env*
  (bvars (list-of symbol?))
  (bvals (list-of reference?))
  (saved-env environment?))

;; apply-env
(extend-env* (bvars bvals saved-env)
  (let ((n (location search-var bvars)))
    (if n
      (list-ref bvals n)
      (apply-env saved-env search-var))))

```

Exercise 4.18

```

;; the-grammar
(expression
  ("letrec"
    (arbno identifier "(" identifier ")" "=" expression)
    "in" expression)
  letrec-exp)

;; value-of
(letrec-exp (p-names b-vars p-bodies letrec-body)
  (value-of letrec-body
    (extend-env-rec* p-names
      b-vars
      p-bodies
      env)))

;; apply-procedure : Proc * ExpVal -> ExpVal
(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)

```

```

                                (value-of body
                                  (extend-env var
                                                (newref val)
                                                saved-env))))))

;; environment datatype
(extend-env-rec*
  (p-names (list-of symbol?))
  (b-vars (list-of symbol?))
  (p-bodies (list-of expression?))
  (saved-env environment?))

;; apply-env
(extend-env-rec* (p-names b-vars p-bodies saved-env)
  (let ((n (location search-var p-names)))
    (if n
      (newref
        (proc-val
          (procedure
            (list-ref b-vars n)
            (list-ref p-bodies n)
            env)))
        (apply-env saved-env search-var))))))

```

Exercise 4.19

```

(define-datatype environment environment?
  (empty-env)
  (extend-env
    (bvar symbol?)
    (bval reference?)
    (saved-env environment?))
  (extend-env-multiproc-rec
    (p-names (list-of symbol?))

```

```

(reference-vector vector?)
(saved-env environment?)))

;; extend-env-rec* :
;; Listof(Var) * Listof(Var) * Listof(Exp) * Env -> Env
(define extend-env-rec*
  (lambda (p-names b-vars p-bodies saved-env)
    (let* ((len (length p-names))
           (reference-vector (make-vector len))
           (new-env (extend-env-multiproc-rec
                      p-names reference-vector saved-env)))
      (for-each (lambda (index b-var body)
                  (vector-set! reference-vector
                               index
                               (newref
                                (proc-val
                                 (procedure
                                  b-var
                                  body
                                  new-env))))))
                (enumerate-interval 0 (- len 1))
                b-vars
                p-bodies)
      new-env)))

;; modify the given pretty-printing procedure
;; env->list : Env -> List
(define env->list
  (lambda (env)
    (cases environment env
      (empty-env () '())
      (extend-env (sym val saved-env)

```



```

      (cons
        (list sym val)
        (env->list saved-env)))
    (extend-env-multiproc-rec (p-names vec saved-env)
      (cons
        (list 'letrec p-names '...)
        (env->list saved-env))))))

```

Exercise 4.20

We choose to not make a new environment constructor, but modify `extend-env` (to allow every instance of `extend-env` in the program to not be rewritten if needed) such that it can contain either references to expressed values or just expressed values. We wrap these types within the denoted values data type.

`apply-env` returns denoted values. Either `ExpVals` or references are wrapped as `DenVals`. Recursive procedures are implicitly mutable since they were not constructed by `let`, hence `extend-env-rec*` returns a wrapped reference. Bound variables are implicitly mutable as well since we do not modify the environment constructor in `apply-procedure`.

There are two places in `value-of` that depend on `apply-env`.

1. `var-exp` calls `apply-env` which returns a denoted value. If the denoted value constructor was wrapped around an `ExpVal`, then that `ExpVal` is returned. If a reference was wrapped, then the reference is dereferenced. In both cases, an `ExpVal` is returned.
2. `assign-exp` calls `apply-env` to determine whether the variable represents an `ExpVal` or a reference. If an `ExpVal` then the variable is immutable. If a reference then the variable is assigned to the new value.

Both `var-exp` and `assign-exp` could determine the type of the value returned from `apply-env` had we not used the `DenVal` data type. But we use `cases` because it is nice.

```

;; the-grammar
(expression
  ("letmutable" identifier "=" expression "in" expression)
  letmutable-exp)

;; value-of
(var-exp (var)
  (cases denval (apply-env env var)
    (denoted-expval (expval) expval)
    (denoted-ref-to-expval (ref) (deref ref))))
(let-exp (var expl body)
  (let ((v1 (value-of expl env)))
    (value-of
      body
      (extend-env var v1 env))))
(letmutable-exp (var expl body)
  (let ((v1 (value-of expl env)))
    (value-of
      body
      (extend-env var (newref v1) env))))
(assign-exp (var expl)
  (cases denval (apply-env env var)
    (denoted-expval (expval)
      (eopl:error
        'value-of
        "cannot assign to immutable var ~s"
        var))
    (denoted-ref-to-expval (ref)
      (begin
        (setref!
          ref
          (value-of expl env))

```

```
(num-val 27))))))
```

```
(define-datatype denval denval?
  (denoted-expval
    (val expval?))
  (denoted-ref-to-expval
    (ref reference?)))

(define-datatype environment environment?
  (empty-env)
  (extend-env
    (bvar symbol?)
    (binding-val (lambda (x) (or expval? reference?)))
    (saved-env environment?))
  (extend-env-rec*
    (proc-names (list-of symbol?))
    (b-vars (list-of symbol?))
    (proc-bodies (list-of expression?))
    (saved-env environment?)))

;; apply-env : Env * Sym -> DenVal
(define apply-env
  (lambda (env search-var)
    (cases environment env
      (empty-env ()
        (eopl:error 'apply-env
          "No binding for ~s"
          search-var))
      (extend-env (bvar bval saved-env)
        (if (eqv? search-var bvar)
          (if (expval? bval)
            (denoted-expval bval)
```

```

        (denoted-ref-to-expval bval))
      (apply-env saved-env search-var)))
    (extend-env-rec* (p-names b-vars p-bodies saved-env)
      (let ((n (location search-var p-names)))
        ;; n : (maybe int)
        (if n
            (denoted-ref-to-expval
              (newref
                (proc-val
                  (procedure
                    (list-ref b-vars n)
                    (list-ref p-bodies n)
                    env))))
            (apply-env saved-env search-var))))))

```

Exercise 4.21

```

;; the-grammar
(expression
  ("setdynamic" identifier "=" expression "during" expression)
  setdynamic-exp)

;; value-of
(setdynamic-exp (var exp1 body)
  (let ((ref (apply-env env var)))
    (let ((original-val (deref ref)))
      (setref! ref (value-of exp1 env))
      (let ((value-of-body (value-of body env)))
        (setref! ref original-val)
        value-of-body))))

```

Exercise 4.22

For `print-stmt` we dispatch on the type of `ExpVal` to unwrap the constructor so the raw value gets printed only. Note there is no `ref-val`

since this language is based on IMPLICIT-REFS.

For `while-stmt` we return `(void)` as the alternative since our Scheme does not allow for no `it` alternatives.

For `decl-blockstmt` we choose to map new locations to the `ExpVal` `(num-val 0)` to represent an uninitialized value. We could modify the store to allow for a new data type that can represent “uninitialized”, although having a representation given by a variant of `ExpVal` would be the simplest. The conflict would be that we may have an unwanted set of expressed values.

We leave all Expression productions in the language because there is use for them. To follow the examples in the text, we allow for multiargument procedures and include addition, subtraction, and `not`.

```
(define the-grammar
  ' ((program (statement) a-program)

    (expression (number) const-exp)
    (expression
      ("-" "(" expression "," expression ")")
      diff-exp)
    (expression
      ("zero?" "(" expression ")")
      zero?-exp)
    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)
    (expression (identifier) var-exp)
    (expression
      ("let" identifier "=" expression "in" expression)
      let-exp)
    (expression
      ("proc" "(" (separated-list identifier ",") ")" expression)
      proc-exp))
```

```

(expression
  ("(" expression (arbno expression) ")")
  call-exp)
(expression
  ("letrec"
    (arbno identifier "(" identifier ")" "=" expression)
    "in" expression)
  letrec-exp)
(expression
  ("begin" expression (arbno ";" expression) "end")
  begin-exp)
(expression
  ("set" identifier "=" expression)
  assign-exp)

;; expression extentions
(expression
  ("+" "(" expression "," expression ")")
  add-exp)
(expression
  ("*" "(" expression "," expression ")")
  mul-exp)
(expression
  ("not" "(" expression ")")
  not-exp)

;; statements
(statement
  (identifier "=" expression)
  assign-stmt)
(statement
  ("print" expression)

```

```

    print-stmt)
(statement
  ("{" (separated-list statement ";") "}")
  block-stmt)
(statement
  ("if" expression statement statement)
  if-stmt)
(statement
  ("while" expression statement)
  while-stmt)
(statement
  ("var" (separated-list identifier ",") ";" statement)
  decl-block-stmt)))

;; result-of-program : Program -> Unspecified
(define result-of-program
  (lambda (pgm)
    (initialize-store!)
    (cases program pgm
      (a-program (stmt)
        (result-of stmt (init-env))))))

;; value-of
(proc-exp (vars body)
  (proc-val (procedure vars body env)))
(call-exp (rator rands)
  (let ((proc (expval->proc (value-of rator env)))
        (args (map (lambda (rand)
                      (value-of rand env))
                    rands)))
    (apply-procedure proc args)))

```

```

;; apply-procedure : Proc * ExpVal -> ExpVal
(define apply-procedure
  (lambda (proc1 vals)
    (cases proc proc1
      (procedure (vars body saved-env)
        (value-of body
          (extend-env* vars
            (map newref vals)
            saved-env))))))

;; result-of : Stmt * Env -> Unspecified
(define result-of
  (lambda (stmt env)
    (cases statement stmt
      (assign-stmt (var exp1)
        (setref! (apply-env env var)
          (value-of exp1 env)))
      (print-stmt (exp1)
        (begin
          (display
            (cases expval (value-of exp1 env)
              (num-val (num) num)
              (bool-val (bool) bool)
              (proc-val (proc) proc)))
            (newline)))
      (block-stmt (stmts)
        (for-each (lambda (stmt)
          (result-of stmt env))
          stmts))
      (if-stmt (exp1 stmt1 stmt2)
        (if (expval->bool (value-of exp1 env))
          (result-of stmt1 env)

```



```

        (result-of stmt2 env)))
(while-stmt (exp1 stmt)
  (let ((vall (value-of exp1 env)))
    (if (expval->bool vall)
        (begin (result-of stmt env)
                 (result-of
                  (while-stmt
                    exp1
                    stmt)
                  env))
        (void))))
(decl-block-stmt (vars stmt)
  (result-of
   stmt
   (extend-env*
    vars
    (map (lambda (v)
           (newref (num-val 0)))
         vars)
   env))))))

(define-datatype proc proc?
  (procedure
   (bvars (list-of symbol?))
   (body expression?)
   (env environment?)))

;; environment datatype
(extend-env*
 (bvars (list-of symbol?))
 (bval (list-of reference?))

```

```
(saved-env environment?))

;; apply-env
(extend-env* (bvars bvals saved-env)
  (let ((n (location search-var bvars)))
    (if n
      (list-ref bvals n)
      (apply-env saved-env search-var))))
```